



En esta práctica, la consigna “especificar en forma completa el TAD...” implica:

- Definir las operaciones y sus parámetros
- Definir los observadores
- Especificar todas las operaciones incluyendo la creación y la igualdad
- Especificar todos los predicados y auxiliares que sean necesarios

## 3.1. TADs introductorios

**Ejercicio 1.** Especificar en forma completa el TAD `NumeroRacional` que incluya las operaciones aritméticas básicas (suma, resta, división, multiplicación) y la operación de *igualdad* que dados dos números racionales devuelva verdadero si son iguales.

**Solución**

```
TAD NumeroRacional {
  obs numerador :  $\mathbb{Z}$ 
  obs denominador :  $\mathbb{Z}$ 

  proc nuevoNumeroRacional(in  $n, d : \mathbb{Z}$ ) : NumeroRacional {
    requiere {  $d \neq 0$  }
    asegura {  $res.numerador = n \wedge res.denominador = d$  }
  }

  proc igualdad(in  $n_1, n_2 : NumeroRacional$ ) : bool {
    asegura {  $res = true \leftrightarrow n_1.numerador \times n_2.denominador = n_2.numerador \times n_1.denominador$  }
  }

  proc suma(in  $n_1, n_2 : NumeroRacional$ ) : NumeroRacional {
    requiere { Verdadero }
    asegura {
       $res.numerador = n_1.numerador \times n_2.denominador + n_2.numerador \times n_1.denominador \wedge$ 
       $res.denominador = n_1.denominador \times n_2.denominador$ 
    }
  }

  proc resta(in  $n_1, n_2 : NumeroRacional$ ) : NumeroRacional {
    requiere { Verdadero }
    asegura {
       $res.numerador = n_1.numerador \times n_2.denominador - n_2.numerador \times n_1.denominador \wedge$ 
       $res.denominador = n_1.denominador \times n_2.denominador$ 
    }
  }

  proc multiplicacion(in  $n_1, n_2 : NumeroRacional$ ) : NumeroRacional {
    requiere { Verdadero }
    asegura {
       $res.numerador = n_1.numerador \times n_2.numerador \wedge$ 
       $res.denominador = n_1.denominador \times n_2.denominador$ 
    }
  }

  proc division(in  $n_1, n_2 : NumeroRacional$ ) : NumeroRacional {
    requiere {  $n_2.numerador \neq 0$  }
    asegura {
```

```

        res.numerador = n1.numerador × n2.denominador ∧
        res.denominador = n1.denominador × n2.numerador
    }
}
}

```

**Ejercicio 2.** Especifique mediante TADs los siguientes elementos geométricos:

a) Punto2D, que representa un punto en el plano. Debe contener las siguientes operaciones:

- *nuevoPunto*: que crea un punto a partir de sus coordenadas  $x$  e  $y$ .
- *mover*: que mueve el punto una determinada distancia sobre los ejes  $x$  e  $y$ .
- *distancia*: que devuelve la distancia entre dos puntos.
- *distanciaAlOrigen*: que devuelve la distancia del punto  $(0,0)$ .

b) Rectángulo2D, que representa un rectángulo en el plano. Debe contener las siguientes operaciones:

- *nuevoRectangulo*: que crea un rectángulo (decida usted cuáles deberían ser los parámetros).
- *mover*: que mueve el rectángulo una determinada distancia en los ejes  $x$  e  $y$ .
- *escalar*: que escala el rectángulo en un determinado factor. Al escalar un rectángulo un punto del mismo debe quedar fijo. En este caso el punto fijo puede ser el centro del rectángulo o uno de sus vértices.
- *estaContenido*: que dados dos rectángulos, indique si uno está contenido en el otro.

c) Punto2D, con las mismas operaciones pero usando como observadores las coordenadas polares

### Solución

```

a) TAD Punto2D {
    obs x : ℝ
    obs y : ℝ

    proc nuevoPunto(in x : ℝ, in y : ℝ) : Punto2D {
        requiere { Verdadero }
        asegura { res.x = x ∧ res.y = y }
    }

    proc igualdad(in P1 : Punto, in P2 : Punto) : bool {
        asegura { res = true ↔ P1.x = P2.x ∧ P1.y = P2.y }
    }

    proc mover(inout p : Punto2D, in dx : ℝ, in dy : ℝ) {
        requiere { p = P0 }
        asegura { p.x = P0.x + dx ∧ p.y = P0.y + dy }
    }

    proc distancia(in p1 : Punto2D, in p2 : Punto2D) : Punto2D {
        requiere { Verdadero }
        asegura { res = distanciaEntrePuntos(p1.x, p1.y, p2.x, p2.y) }
    }

    aux distanciaEntrePuntos(x1 : ℝ, y1 : ℝ, x2 : ℝ, y2 : ℝ) : ℝ = √((x2 - x1)² + (y2 - y1)²)

    proc distanciaAlOrigen(in p : Punto2D) : ℝ {

```

```

    requiere { Verdadero }
    asegura { res = distanciaEntrePuntos(p.x, p.y, 0, 0) }
  }
}

```

b) TAD Rectangulo2D {

– *Posición del vértice inferior izquierdo*

obs posición : struct  $\langle x : \mathbb{R}, y : \mathbb{R} \rangle$

– *Tamaño del rectángulo (ancho y alto hacia ejes positivos)*

obs tamaño : struct  $\langle dx : \mathbb{R}, dy : \mathbb{R} \rangle$

proc nuevoRectangulo(in pos : tupla  $\langle \mathbb{R}, \mathbb{R} \rangle$ , in tam : tupla  $\langle \mathbb{R}, \mathbb{R} \rangle$ ) : Rectangulo2D {

  requiere { tam.x  $\geq 0 \wedge$  tam.y  $\geq 0$  }

  asegura { res.posición = pos  $\wedge$  res.tamaño = tam }

}

proc igualdad(in  $R_1$  : Rectangulo2D, in  $R_2$  : Rectangulo2D) : bool {

  asegura { res = true  $\leftrightarrow R_1$ .posición =  $R_2$ .posición  $\wedge R_1$ .tamaño =  $R_2$ .tamaño }

}

proc mover(inout r : Rectangulo2D, in dxy : tupla  $\langle \mathbb{R}, \mathbb{R} \rangle$ ) {

  requiere {  $r = R_0$  }

  asegura {

*$r$ .tamaño =  $R_0$ .tamaño*

*$r$ .posición.x =  $R_0$ .posición.x + dxy.x  $\wedge$   $r$ .posición.y =  $R_0$ .posición.y + dxy.y*

  }

}

proc escalar(inout r : Rectangulo2D, in esc :  $\mathbb{R}$ ) {

  requiere {  $r = R_0$  }

  asegura {

*$r$ .posición =  $R_0$ .posición*

*$r$ .tamaño.x =  $R_0$ .tamaño.x \* esc  $\wedge$   $r$ .tamaño.y =  $R_0$ .tamaño.y \* esc*

  }

}

proc estaContenido(in  $r_1$  : Rectangulo2D, in  $r_2$  : Rectangulo2D) : bool {

  requiere { Verdadero }

  asegura {

    res = true  $\leftrightarrow$  (

$r_1$ .posición.x  $\geq r_2$ .posición.x  $\wedge r_1$ .posición.y  $\geq r_2$ .posición.y  $\wedge$

$r_1$ .posición.x +  $r_1$ .tamaño.x  $\leq r_2$ .posición.x +  $r_2$ .tamaño.x  $\wedge$

$r_1$ .posición.y +  $r_1$ .tamaño.y  $\leq r_2$ .posición.y +  $r_2$ .tamaño.y

    )

  }

}

}

c) TAD Punto2D {

  obs r :  $\mathbb{R}$

  obs  $\theta$  :  $\mathbb{R}$

  proc nuevoPunto(in x, y :  $\mathbb{R}$ ) : Punto2D {

```

    requiere { Verdadero }
    asegura { res.r = radio(x,y) ∧ esAngulo(res.θ,x,y) }
}

proc igualdad(in P1 : Punto2D, in P2 : Punto2D) : bool {
    asegura { res = true ↔ (P1.r = P2.r ∧ (∃k : ℤ) (P1.θ = P2.θ + 2kπ)) }
}

proc mover(inout p : Punto2D, dx, dy : ℝ) {
    requiere { p = P0 }
    asegura {
        p.r = radio(coordX(P0.r, P0.θ) + dx, coordY(P0.r, P0.θ) + dy) ∧
        esAngulo(p.θ, coordX(P0.r, P0.θ) + dx, coordY(P0.r, P0.θ) + dy)
    }
}

proc distancia(in p1, p2 : Punto2D) : ℝ {
    requiere { Verdadero }
    asegura { res = distanciaEntre(p1, p2) }
}

proc distanciaAlOrigen(in p : Punto2D) : ℝ {
    requiere { Verdadero }
    asegura { res = p.r }
}

aux radio(x, y : ℝ) : ℝ =  $\sqrt{x^2 + y^2}$ 
aux angulo(x, y : ℝ) : ℝ = atan(y/x)
pred esAngulo(θ, x, y : ℝ) {
    (x ≠ 0 →L θ = angulo(x, y)) ∧
    (x = 0 → ((y >= 0 → θ = π/2) ∧ (y < 0 → θ = 3 * π/2)))
}

aux coordX(r, θ : ℝ) : ℝ = r * cos(θ)
aux coordY(r, θ : ℝ) : ℝ = r * sin(θ)
aux distanciaEntre(p1, p2 : Punto2D) : ℝ =
     $\sqrt{(coordX(p2.r, p2.θ) - coordX(p1.r, p1.θ))^2 + (coordY(p2.r, p2.θ) - coordY(p1.r, p1.θ))^2}$ 
}

```

### 3.2. TADs sobre secuencias

Resuelva los ejercicios de esta sección usando como observador alguna  $\text{seq}\langle T \rangle$

#### Ejercicio 3.

a) Especifique el TAD Cola $\langle T \rangle$  con las siguientes operaciones:

- *nuevaCola* que crea una cola vacía
- *estáVacía* que devuelve true si la cola no contiene elementos
- *encolar* que agrega un elemento al final de la cola
- *desencolar* que elimina el primer elemento de la cola y lo devuelve

b) Especifique el TAD Pila $\langle T \rangle$  con las siguientes operaciones

- *nuevaPila* que crea una pila vacía
  - *estáVacía* que devuelve true si la pila no contiene elementos
  - *apilar* que agrega un elemento al tope de la pila
  - *desapilar* que elimina el elemento del tope de la pila y lo devuelve
- c) Especifique el TAD *DobleCola* $\langle T \rangle$ , en el que los elementos pueden insertarse al principio o al final y se eliminan por el medio. Debe contener las operaciones
- *nuevaDobleCola* que crea una cola vacía
  - *estáVacía* que devuelve true si la cola no contiene elementos
  - *encolarAtrás* que agrega un elemento al final de la cola
  - *encolarAdelante* que agrega un elemento al principio de la cola
  - *desencolar* que elimina el primer elemento de la cola y lo devuelve

Ejemplo:

```
c:= new NuevaDobleCola<T>()

encolarAdelante(c, 1)    // c = <1>
encolarAdelante(c, 2)    // c = <2,1>
encolarAtrás(c, 3)      // c = <2,1,3>

desencolar(c)           // devuelve 1, c = <2,3>
desencolar(c)           // devuelve 2, c = <3>
desencolar(c)           // devuelve 3, c = <>
```

### Solución

```
a) TAD Cola<T> {
  obs elems : seq<T>
  proc nuevaCola() : Cola<T> {
    requiere { Verdadero }
    asegura { |res.elems| = 0 }
  }

  proc igualdad(in c1, c2 : Cola<T>) : bool {
    asegura { res  $\leftrightarrow$  c1.elems = c2.elems }
  }

  proc estaVacía(in c : Cola<T>) : bool {
    requiere { Verdadero }
    asegura { res = True  $\leftrightarrow$  |c.elems| = 0 }
  }

  proc encolar(inout c : Cola<T>, in e : T) {
    requiere { c = C0 }
    asegura { c.elems = C0.elems ++ <e> }
  }

  proc desencolar(inout c : Cola<T>) : T {
    requiere { c = C0  $\wedge$  |c.elems| > 0 }
    asegura { res = C0.elems[0]  $\wedge$  sacoElPrimero(C0, c) }
  }
}
```

```

}

pred sacoElPrimero( $C_0, c : \text{Cola}\langle T \rangle$ ) {
   $|c.elems| = |C_0.elems| - 1 \wedge_L$ 
   $(\forall i : \mathbb{Z}) (0 \leq i < |c.elems| \rightarrow_L c.elems[i] = C_0.elems[i + 1])$ 
}

}

```

b) TAD Pila $\langle T \rangle$  {

```

  obs elems : seq $\langle T \rangle$ 
  proc nuevaPila() : Pila $\langle T \rangle$  {
    requiere { Verdadero }
    asegura {  $|res.elems| = 0$  }
  }

  proc igualdad(in  $p_1, p_2 : \text{Pila}\langle T \rangle$ ) : bool {
    asegura {  $res \leftrightarrow p_1.elems = p_2.elems$  }
  }

  proc estaVacía(in  $c : \text{Pila}\langle T \rangle$ ) : bool {
    requiere { Verdadero }
    asegura {  $res = \text{True} \leftrightarrow |c.elems| = 0$  }
  }

  proc apilar(inout  $c : \text{Pila}\langle T \rangle$ , in  $e : T$ ) {
    requiere {  $p = P_0$  }
    asegura {  $\text{agregadoAdelante}(P_0, p, e)$  }
  }

  proc desapilar(inout  $c : \text{Pila}\langle T \rangle$ ) :  $T$  {
    requiere {  $p = P_0 \wedge |p.elems| > 0$  }
    asegura {  $res = P_0.elems[0] \wedge \text{sacoElPrimero}(P_0, p)$  }
  }

  pred agregadoAdelante( $P_0, p : \text{Pila}\langle T \rangle, e : T$ ) {
     $p.elems[0] = e \wedge |p.elems| = |P_0.elems| + 1 \wedge_L$ 
     $(\forall i : \mathbb{Z}) (1 \leq i < |p.elems| \rightarrow_L p.elems[i] = P_0.elems[i - 1])$ 
  }

  pred sacoElPrimero( $P_0, p : \text{Pila}\langle T \rangle$ ) {
     $|p.elems| = |P_0.elems| - 1 \wedge_L$ 
     $(\forall i : \mathbb{Z}) (0 \leq i < |p.elems| \rightarrow_L p.elems[i] = P_0.elems[i + 1])$ 
  }
}

```

c) TAD DobleCola $\langle T \rangle$  {

```

  obs elems : seq $\langle T \rangle$ 
  proc nuevaDobleCola() : DobleCola $\langle T \rangle$  {
    requiere { Verdadero }
    asegura {  $|res.elems| = 0$  }
  }

  proc igualdad(in  $c_1, c_2 : \text{DobleCola}\langle T \rangle$ ) : bool {

```

```

    asegura { res  $\leftrightarrow$   $c_1.\text{elems} = c_2.\text{elems}$  }
  }

proc estaVacia(in c : DobleCola<T>) : bool {
  requiere { Verdadero }
  asegura { res = True  $\leftrightarrow$   $|c.\text{elems}| = 0$  }
}

proc encolarAtrás(inout c : DobleCola<T>, in elem : T) {
  requiere {  $c = C_0$  }
  asegura {  $c.\text{elems} = C_0.\text{elems} ++ \langle e \rangle$  }
}

proc encolarAdelante(inout c : DobleCola<T>, in elem : T) {
  requiere {  $p = P_0$  }
  asegura { agregadoAdelante( $C_0, c, e$ ) }
}

– Tomo  $\div$  como división entera
proc desencolar(inout c : DobleCola<T>) : T {
  requiere {  $c = C_0 \wedge |c.\text{elems}| > 0$  }
  asegura { res =  $C_0.\text{elems}[|C_0.\text{elems}| \div 2] \wedge \text{sacoElDelMedio}(C_0, c)$  }
}

pred agregadoAdelante( $C_0, c : \text{DobleCola}\langle T \rangle, e : T$ ) {
   $c.\text{elems}[0] = e \wedge |c.\text{elems}| = |C_0.\text{elems}| + 1 \wedge_L$ 
   $(\forall i : \mathbb{Z}) (0 \leq i < |c.\text{elems}| \rightarrow_L c.\text{elems}[i] = C_0.\text{elems}[i - 1])$ 
}

pred sacoElDelmedio( $C_0, c : \text{DobleCola}\langle T \rangle$ ) {
   $|c.\text{elems}| = |C_0.\text{elems}| - 1 \wedge_L ($ 
   $(\forall i : \mathbb{Z}) (0 \leq i < |c.\text{elems}| \div 2 \rightarrow_L c.\text{elems}[i] = C_0.\text{elems}[i]) \wedge$ 
   $(\forall i : \mathbb{Z}) (|c.\text{elems}| \div 2 \leq i < |c.\text{elems}| \rightarrow_L c.\text{elems}[i] = C_0.\text{elems}[i + 1])$ 
  )
}

```

#### Ejercicio 4.

a) Especifique el TAD  $\text{Diccionario}\langle K, V \rangle$  con las siguientes operaciones:

- *nuevoDiccionario*: que crea un diccionario vacío
- *definir*: que agrega un par clave-valor al diccionario
- *obtener*: que devuelve el valor asociado a una clave
- *está*: que devuelve true si la clave está en el diccionario
- *borrar*: que elimina una clave del diccionario

b) Especifique el TAD  $\text{DiccionarioConHistoria}\langle K, V \rangle$ . El mismo permite consultar, para cada clave, todos los valores que se asociaron con la misma a lo largo del tiempo (en orden). Se debe poder hacer dicha consulta aún si la clave fue borrada.

#### Solución

```

a) TAD Diccionario $\langle K, V \rangle$  {
  obs elems : seq< tupla  $\langle K, V \rangle$  >
  proc nuevoDiccionario() : Diccionario $\langle K, V \rangle$  {
    requiere { Verdadero }
    asegura { res.elems =  $\langle \rangle$  }
  }

  proc igualdad(in  $d_1, d_2$  : Diccionario $\langle K, V \rangle$ ) : bool {
    asegura { res  $\leftrightarrow d_1.elems = d_2.elems$  }
  }

  proc definir(inout  $d$  : Diccionario $\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {
    requiere {  $d = D_0$  }
    asegura {
      existeClave( $k, D_0$ )  $\wedge$  cambiaValor( $D_0, d, k, v$ )  $\vee$ 
       $\neg$  existeClave( $k, D_0$ )  $\wedge$  claveDefinida( $D_0, d, k, v$ )
    }
  }

  pred existeClave( $d$  : Diccionario $\langle K, V \rangle$ ,  $k : K$ ) {
    ( $\exists i : \mathbb{Z}$ ) ( $0 \leq i < |d.elems| \wedge_L d.elems[i]_0 = k$ )
  }

  pred cambiaValor( $D_0$  : Diccionario $\langle K, V \rangle$ ,  $d$  : Diccionario $\langle K, V \rangle$ ,  $k : K, v : V$ ) {
    mismasClaves( $D_0, d$ )  $\wedge$  cambiaSoloK( $D_0, d, k, v$ )
  }

  pred mismasClaves( $D_0$  : Diccionario $\langle K, V \rangle$ ,  $d$  : Diccionario $\langle K, V \rangle$ ) {
     $|D_0.elems| = |d.elems| \wedge$ 
    ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |D_0.elems| \rightarrow_L D_0.elems[i]_0 = d.elems[i]_0$ )
  }

  pred cambiaSoloK( $D_0$  : Diccionario $\langle K, V \rangle$ ,  $d$  : Diccionario $\langle K, V \rangle$ ,  $k : K, v : V$ ) {
    ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |D_0.elems| \rightarrow_L$ 
       $D_0.elems[i]_0 = k \wedge d.elems[i]_1 = v \vee$ 
       $D_0.elems[i]_0 \neq k \wedge d.elems[i]_1 = D_0.elems[i]_1$ )
  }

  pred claveDefinida( $D_0$  : Diccionario $\langle K, V \rangle$ ,  $d$  : Diccionario $\langle K, V \rangle$ ,  $k : K, v : V$ ) {
     $d.elems = D_0.elems ++ \langle \langle k, v \rangle \rangle$ 
  }

  proc obtener(in  $d$  : Diccionario $\langle K, V \rangle$ , in  $k : K$ ) :  $V$  {
    requiere { existeClave( $d, k$ ) }
    asegura { ( $\exists i : \mathbb{Z}$ ) ( $0 \leq i < |d.elems| \wedge_L d.elems[i]_0 = k \wedge res = d.elems[i]_1$ ) }
  }

  proc esta(in  $d$  : Diccionario $\langle K, V \rangle$ , in  $k : K$ ) : bool {
    requiere { Verdadero }
    asegura { res = true  $\leftrightarrow$  existeClave( $d, k$ ) }
  }

```

```

proc borrar(inout  $d : \text{Diccionario}\langle K, V \rangle$ , in  $k : K$ ) {
  requiere {  $d = D_0 \wedge \text{existeClave}(d, k)$  }
  asegura {  $\text{sinClave}(D_0, d, k)$  }
}

pred sinClave( $D_0 : \text{Diccionario}\langle K, V \rangle$ ,  $d : \text{Diccionario}\langle K, V \rangle$ ,  $k : K$ ) {
   $|d.\text{elems}| = |D_0.\text{elems}| - 1 \wedge$ 
   $(\exists i : \mathbb{Z}) (0 \leq i < |D_0.\text{elems}| \wedge_L$ 
     $D_0.\text{elems}[i]_0 = k \wedge$ 
     $(\forall j_1 : \mathbb{Z}) (0 \leq j_1 < i \rightarrow_L d.\text{elems}[j_1] = D_0.\text{elems}[j_1]) \wedge$ 
     $(\forall j_2 : \mathbb{Z}) (i < j_2 < |d| \rightarrow_L d.\text{elems}[j_2] = D_0.\text{elems}[j_2 + 1])$ 
  )
}

```

b) TAD  $\text{DiccionarioConHistoria}\langle K, V \rangle$  {

```

  obs elems : seq< tupla  $\langle K, V \rangle$  >
  obs historial : seq< tupla  $\langle K, \text{seq}\langle V \rangle$  > >

  proc nuevoDiccionarioConHistoria() :  $\text{DiccionarioConHistoria}\langle K, V \rangle$  {
    requiere { Verdadero }
    asegura {  $\text{res.elems} = \langle \rangle \wedge \text{res.historial} = \langle \rangle$  }
  }

  proc igualdad(in  $d_1, d_2 : \text{DiccionarioConHistoria}\langle K, V \rangle$ ) : bool {
    requiere { Verdadero }
    asegura {  $d_1.\text{elems} = d_2.\text{elems} \wedge d_1.\text{historial} = d_2.\text{historial}$  }
  }

  proc definir(inout  $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {
    requiere {  $d = D_0$  }
    asegura {
       $\text{existeClave}(k, D_0) \wedge \text{cambiaValor}(D_0, d, k, v) \vee$ 
       $\neg \text{existeClave}(k, D_0) \wedge \text{claveDefinida}(D_0, d, k, v)$ 
    }
  }

  pred existeClave( $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ ,  $k : K$ ) {
     $(\exists i : \mathbb{Z}) (0 \leq i < |d.\text{elems}| \wedge_L d.\text{elems}[i]_0 = k)$ 
  }

  pred cambiaValor( $D_0 : \text{DiccionarioConHistoria}\langle K, V \rangle$ ,  $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ ,  $k : K$ ,  $v : V$ ) {
     $\text{mismasClaves}(D_0, d) \wedge \text{cambiaSoloK}(D_0, d, k, v)$ 
  }

  pred mismasClaves( $D_0 : \text{DiccionarioConHistoria}\langle K, V \rangle$ ,  $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ ) {
     $|D_0.\text{elems}| = |d.\text{elems}| \wedge |D_0.\text{historial}| = |d.\text{historial}| \wedge$ 
     $(\forall i : \mathbb{Z}) (0 \leq i < |D_0.\text{elems}| \rightarrow_L D_0.\text{elems}[i]_0 = d.\text{elems}[i]_0) \wedge$ 
     $(\forall i : \mathbb{Z}) (0 \leq i < |D_0.\text{historial}| \rightarrow_L D_0.\text{historial}[i]_0 = d.\text{historial}[i]_0)$ 
  }

```

```

pred cambiaSoloK( $D_0 : \text{DiccionarioConHistoria}\langle K, V \rangle, d : \text{DiccionarioConHistoria}\langle K, V \rangle, k : K, v : V$ ) {
  ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |D_0.\text{elems}| \rightarrow_L$ 
     $D_0.\text{elems}[i]_0 = k \wedge d.\text{elems}[i]_1 = v \vee$ 
     $D_0.\text{elems}[i]_0 \neq k \wedge d.\text{elems}[i]_1 = D_0.\text{elems}[i]_1$ )
   $\wedge$ 
  ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |D_0.\text{historial}| \rightarrow_L$ 
     $D_0.\text{historial}[i]_0 = k \wedge d.\text{historial}[i]_1 = D_0.\text{historial}[i]_1 ++ \langle v \rangle \vee$ 
     $D_0.\text{historial}[i]_0 \neq k \wedge d.\text{historial}[i]_1 = D_0.\text{historial}[i]_1$ )
}

– La clave se puede haber borrado del diccionario pero seguir en el historial
pred claveDefinida( $D_0 : \text{DiccionarioConHistoria}\langle K, V \rangle, d : \text{DiccionarioConHistoria}\langle K, V \rangle, k : K, v : V$ ) {
   $d.\text{elems} = D_0.\text{elems} ++ \langle \langle k, v \rangle \rangle \wedge$ 
   $\neg \text{estaEnHistorial}(D_0, k) \wedge d.\text{historial} = D_0.\text{historial} ++ \langle \langle k, \langle v \rangle \rangle \rangle \vee$ 
   $\text{estaEnHistorial}(D_0, k) \wedge (\forall i : \mathbb{Z}) (0 \leq i < |D_0.\text{historial}| \rightarrow_L$ 
     $D_0.\text{historial}[i]_0 = k \wedge d.\text{historial}[i]_1 = D_0.\text{historial}[i]_1 ++ \langle v \rangle \vee$ 
     $D_0.\text{historial}[i]_0 \neq k \wedge d.\text{historial}[i]_1 = D_0.\text{historial}[i]_1$ )
}

pred estaEnHistorial( $d : \text{DiccionarioConHistoria}\langle K, V \rangle, k : K$ ) {
  ( $\exists i : \mathbb{Z}$ ) ( $0 \leq i < |d.\text{historial}| \wedge_L d.\text{historial}[i]_0 = k$ )
}

proc obtener(in  $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ , in  $k : K$ ) :  $V$  {
  requiere {  $\text{existeClave}(d, k)$  }
  asegura { ( $\exists i : \mathbb{Z}$ ) ( $0 \leq i < |d.\text{elems}| \wedge_L d.\text{elems}[i]_0 = k \wedge \text{res} = d.\text{elems}[i]_1$ ) }
}

proc esta(in  $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ , in  $k : K$ ) : bool {
  requiere { Verdadero }
  asegura {  $\text{res} = \text{true} \leftrightarrow \text{existeClave}(d, k)$  }
}

proc borrar(inout  $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ , in  $k : K$ ) {
  requiere {  $d = D_0 \wedge \text{existeClave}(d, k)$  }
  asegura {  $\text{sinClave}(D_0, d, k) \wedge d.\text{historial} = D_0.\text{historial}$  }
}

pred sinClave( $D_0 : \text{DiccionarioConHistoria}\langle K, V \rangle, d : \text{DiccionarioConHistoria}\langle K, V \rangle, k : K$ ) {
   $|d.\text{elems}| = |D_0.\text{elems}| - 1 \wedge$ 
  ( $\exists i : \mathbb{Z}$ ) ( $0 \leq i < |D_0.\text{elems}| \wedge_L$ 
     $D_0.\text{elems}[i]_0 = k \wedge$ 
    ( $\forall j_1 : \mathbb{Z}$ ) ( $0 \leq j_1 < i \rightarrow_L d.\text{elems}[j_1] = D_0.\text{elems}[j_1]$ )  $\wedge$ 
    ( $\forall j_2 : \mathbb{Z}$ ) ( $i < j_2 < |d| \rightarrow_L d.\text{elems}[j_2] = D_0.\text{elems}[j_2 + 1]$ )
  )
}

proc obtenerHistorial(in  $d : \text{DiccionarioConHistoria}\langle K, V \rangle$ , in  $k : K$ ) :  $\text{seq}\langle V \rangle$  {
  requiere {  $\text{estaEnHistorial}(d, k)$  }
  asegura { ( $\exists i : \mathbb{Z}$ ) ( $0 \leq i < |d.\text{historial}| \wedge_L d.\text{historial}[i]_0 = k \wedge \text{res} = d.\text{historial}[i]_1$ ) }
}

```

**Ejercicio 5.** Especifique el TAD  $\text{Conjunto}\langle T \rangle$  con las operaciones

- *nuevoConjunto*: que crea un conjunto vacío
- *agregar*: que agrega un elemento al conjunto
- *pertenece*: que devuelve true si un elemento pertenece al conjunto
- *eliminar*: que elimina un elemento del conjunto

### Solución

```

TAD Conjunto<T> {
  obs elems : seq<T>
  proc conjuntoVacio() : Conjunto<T> {
    requiere { Verdadero }
    asegura { |res.elems| = 0 }
  }

  proc igualdad(in c1, c2 : Conjunto<T>) : bool {
    requiere { Verdadero }
    asegura { res = true ↔ (∀e : T) (e ∈ c1.elems ↔ e ∈ c2.elems) }
  }

  proc agregar(inout c : Conjunto<T>, in e : T) {
    requiere { c = C0 }
    asegura {
      e ∈ c.elems ∧
      siguenLosMismos(C0, c) ∧
      soloSeAgregó(C0, c, e)
    }
  }

  pred siguenLosMismos(C0, c : Conjunto<T>) {
    (∀t : T) (t ∈ C0.elems → t ∈ c.elems)
  }

  pred soloSeAgregó(C0, c : Conjunto<T>, e : T) {
    (∀t : T) (t ∈ c.elems ∧ t ≠ e → t ∈ C0.elems)
  }

  proc pertenece(in c : Conjunto<T>, in e : T) : bool {
    requiere { Verdadero }
    asegura { res = true ↔ e ∈ c.elems }
  }

  proc eliminar(inout c : Conjunto<T>, in e : T) {
    requiere { c = C0 }
    asegura {
      ¬(e ∈ c.elems) ∧
      noSaquéOtros(C0, c, e)
    }
  }

  pred noSaquéOtros(C0, c : Conjunto<T>, e : T) {
    (∀t : T) (t ≠ e → (t ∈ C0.elems ↔ t ∈ c.elems))
  }
}

```

### 3.3. TADs sobre conjuntos y diccionarios

**Ejercicio 6.** Especifique los siguientes TADs usando como observador algún  $\text{conj}\langle T \rangle$  (con las operaciones descritas en la sección anterior). **Nota:** Pensar bien qué  $T$  elegir

- Diccionario< $K, V$ >
- Pila< $P$ >

```

a) TAD Diccionario⟨K, V⟩ {
  obs elems : conj⟨struct ⟨clave : K, valor : V⟩⟩
  proc nuevoDiccionario() : Diccionario⟨K, V⟩ {
    requiere { Verdadero }
    asegura { res.elems = ⟨⟩ }
  }

  proc igualdad(in d1, d2 : Diccionario⟨K, V⟩) : bool {
    asegura {
      res = true ↔
      (∀s : struct ⟨clave : K, valor : V⟩) (s ∈ d1.elems ↔ t ∈ d2.elems)
    }
  }

  proc definir(inout d : Diccionario⟨K, V⟩, in k : K, in v : V) {
    requiere { d = D0 }
    asegura {
      ⟨k, v⟩ ∈ d.elems ∧
      noCambianOtras(D0, d, k) ∧L
      cantApariciones(d, k) = 1
    }
  }

  pred noCambiaronOtras(D0, d : Diccionario⟨K, V⟩, k : K) {
    (∀s : struct ⟨clave : K, valor : V⟩) (s.clave ≠ k → (s ∈ D0.elems ↔ s ∈ d.elems))
  }

  aux cantApariciones(d : Diccionario⟨K, V⟩, k : K) : ℤ =
    ∑s:struct ⟨clave:K, valor:V⟩ IfThenElse(s.clave = k, 1, 0)

  proc obtener(in d : Diccionario⟨K, V⟩, in k : K) : V {
    requiere { (∃s : struct ⟨clave : K, valor : V⟩) (s ∈ d.elems ∧ s.clave = k) }
    asegura { (∃s : struct ⟨clave : K, valor : V⟩) (s ∈ d.elems ∧ s.clave = k ∧ res = s.valor) }
  }

  proc está(in d : Diccionario⟨K, V⟩, in k : K) : bool {
    requiere { Verdadero }
    asegura { res = True ↔ (∃s : struct ⟨clave : K, valor : V⟩) (s ∈ d.elems ∧ s.clave = k) }
  }

  proc borrar(inout d : Diccionario⟨K, V⟩, in k : K) {
    requiere { d = D0 ∧ (∃s : struct ⟨clave : K, valor : V⟩) (s ∈ d.elems ∧ s.clave = k) }
    asegura { ⟨k, v⟩ ∉ d.elems ∧ noCambianOtras(D0, d, k) }
  }
}

b) TAD Pila⟨T⟩ {
  obs elemsEnPosicion : conj⟨struct ⟨valor⟩T, posicion : ℤ⟩

```

```

proc nuevaPila() : Pila⟨T⟩ {
  requiere { Verdadero }
  asegura { res.elemsEnPosicion = ⟨⟩ }
}

proc igualdad(in p1, p2 : Pila⟨T⟩) : bool {
  asegura { res ↔ p1.elemsEnPosicion = p2.elemsEnPosicion }
}

proc estaVacía(in p : Pila⟨T⟩) : bool {
  requiere { Verdadero }
  asegura { res = True ↔ p.elemsEnPosicion = ⟨⟩ }
}

proc apilar(inout p : Pila⟨T⟩, in e : T) {
  requiere { p = P0 }
  asegura { apilado(P0, p, e) }
}

proc desapilar(inout p : Pila⟨T⟩) : T {
  requiere { p = P0 ∧ |p.elemsEnPosicion| > 0 }
  asegura { esTope(P0, res) ∧ desapilado(P0, p) }
}

pred apilado(P0, p : Pila⟨T⟩, e : T) {
  (∃i : ℤ) (
    esProximaPosicion(P0, k) ∧ p.elemsEnPosicion = P0 ∪ ⟨⟨valor : e, posicion : i⟩⟩
  )
}

pred esProximaPosicion(p : Pila⟨T⟩, i : ℤ) {
  (p.elemsEnPosicion = ⟨⟩ ∧ i = 0) ∨L ((∃j : ℤ) (esPosicionDeTope(p, j) ∧ i = j + 1))
}

pred esPosicionDeTope(p : Pila⟨T⟩, i : ℤ) {
  (∀s : struct ⟨valor : T, posicion : ℤ⟩) (s ∈ p.elemsEnPosicion →L s.posicion ≤ i)
}

pred esTope(p : Pila⟨T⟩, res : T) {
  (∃s : struct ⟨valor : T, posicion : ℤ⟩) (
    s ∈ p.elemsEnPosicion ∧L res = s.valor ∧ esPosicionDeTope(p, s.posicion)
  )
}

pred desapilado(P0, p : Pila⟨T⟩) {
  (∃s : struct ⟨valor : T, posicion : ℤ⟩) (
    s ∈ P0.elemsEnPosicion ∧
    esPosicionDeTope(P0, s.posicion) ∧
    p.elemsEnPosicion = P0.elemsEnPosicion - ⟨t⟩
  )
}

```

**Ejercicio 7.** Especificar el TAD *Secuencia* $\langle S \rangle$  usando como observador algún *conj* $\langle T \rangle$  con las siguientes operaciones

- *nuevaSecuencia* Crea una secuencia vacía
- *vacía* Devuelve si la secuencia tiene algún elemento
- *longitud* Devuelve la cantidad de elementos de la secuencia
- *obtener* Devuelve el elemento en la posición  $i$
- *agregarAdelante* Agrega un elemento al comienzo de la secuencia (en la posición 0)
- *agregarAtrás* Agrega un elemento al final de la secuencia
- *modificarPosicion* Modifica el elemento en la posición  $i$
- *concatenar* Toma dos secuencias  $s_1$  y  $s_2$  y genera una con los elementos de  $s_1$  seguidos por los de  $s_2$ <sup>1</sup>

### Solución

```

TAD Secuencia $\langle T \rangle$  {
  obs elemsEnPosicion : conj<struct  $\langle elem : T, pos : \mathbb{Z} \rangle$ >
  proc nuevaSecuencia() : Secuencia $\langle T \rangle$  {
    requiere { Verdadero }
    asegura { res.elemsEnPosicion =  $\langle \rangle$  }
  }

  proc vacía(in s : Secuencia $\langle T \rangle$ ) : bool {
    requiere { Verdadero }
    asegura { res = true  $\leftrightarrow$  s.elemsEnPosicion =  $\langle \rangle$  }
  }

  proc longitud(in s : Secuencia $\langle T \rangle$ ) :  $\mathbb{Z}$  {
    requiere { Verdadero }
    asegura { res = |s.elemsEnPosicion| }
  }

  proc obtener(in s : Secuencia $\langle T \rangle$ , in i :  $\mathbb{Z}$ ) : T {
    requiere { indiceEnRango(s, i) }
    asegura {  $\langle elem : res, pos : i \rangle \in s.elemsEnPosicion$  }
  }

  proc agregarAdelante(inout s : Secuencia $\langle T \rangle$ , in e : T) {
    requiere { s =  $S_0$  }
    asegura { |s| = | $S_0$ | + 1  $\wedge \langle elem : e, pos : 0 \rangle \in s \wedge$  mismosElementosCorridosUno(s,  $S_0$ ) }
  }

  proc agregarAtras(inout s : Secuencia $\langle T \rangle$ , in e : T) {
    requiere { s =  $S_0$  }
    asegura { s.elemsEnPosicion =  $S_0.elemsEnPosicion \cup \{ \langle elem : e, pos : |s.elemsEnPosicion| \rangle \}$  }
  }

  proc modificarPosicion(inout s : Secuencia $\langle T \rangle$ , in i :  $\mathbb{Z}$ , in e : T) {
    requiere { s =  $S_0 \wedge$  indiceEnRango(s, i) }
    asegura {
      |s| = | $S_0$ |  $\wedge (\forall st : \text{struct } \langle elem : T, pos : \mathbb{Z} \rangle) ($ 
        st  $\in S_0.elemsEnPosicion \rightarrow \text{IfThenElse}(st.pos = i, \langle elem : e, pos : i \rangle, st) \in s.elemsEnPosicion$ 
      )
    }
  }

  proc concatenar(in s, t : Secuencia $\langle T \rangle$ ) : Secuencia $\langle T \rangle$  {
    requiere { Verdadero }
    asegura {
      |res.elemsEnPosicion| = |s.elemsEnPosicion| + |t.elemsEnPosicion|  $\wedge$ 
      mismosElementosCorridos(res, s, 0)  $\wedge$  mismosElementosCorridos(res, t, |s.elemsEnPosicion|)
    }
  }
}

```

<sup>1</sup>Puede modificar  $s_1$  o devolver una secuencia nueva

```

}
proc igualdad(in s, t : Secuencia⟨T⟩) : bool {
  requiere { Verdadero }
  asegura { res = true ↔ s.elemEnPosicion = t.elemEnPosicion }
}
pred indiceEnRango(s : Secuencia⟨T⟩, i : ℤ) {
  0 ≤ i < |s.elemEnPosicion|
}
pred mismosElementosCorridos(s, S0 : Secuencia⟨T⟩, k : ℤ) {
  (∀ e : struct ⟨elem : T, pos : ℤ⟩) (
    e ∈ S0.elemEnPosicion → ⟨elem : e.elem, pos : e.pos + k⟩ ∈ s.elemEnPosicion
  )
}
pred mismosElementosCorridosUno(s, S0 : Secuencia⟨T⟩) { mismosElementosCorridos(s, S0, 1) }
}

```

**Ejercicio 8.** Especificar los siguientes TADs usando como observador algún  $\text{dicc}\langle K, V \rangle$

- a) Pila⟨P⟩
- b) Secuencia⟨S⟩
- c) Conjunto⟨C⟩
- d) DobleCola⟨D⟩

**Solución**

- a) Nótese que este observador no permite apilar más de una vez el mismo  $T$

```

TAD Pila⟨T⟩ {
  obs elemEnPosicion : dicc⟨T, ℤ⟩
  proc nuevaPila() : Pila⟨T⟩ {
    requiere { Verdadero }
    asegura { res.elemEnPosicion = ⟨⟩ }
  }
  proc igualdad(in p1, p2 : Pila⟨T⟩) : bool {
    asegura { res ↔ p1.elemEnPosicion = p2.elemEnPosicion }
  }
  proc estaVacía(in p : Pila⟨T⟩) : bool {
    requiere { Verdadero }
    asegura { res ↔ res.elemEnPosicion = ⟨⟩ }
  }
  proc apilar(inout p : Pila⟨T⟩, in e : T) {
    requiere { p = P0 }
    asegura { apilado(P0, p, e) }
  }
  proc desapilar(inout p : Pila⟨T⟩) : T {
    requiere { p = P0 ∧ |p.elemEnPosicion| > 0 }
    asegura { esTope(P0, res) ∧ desapilado(P0, p) }
  }
}

```

```

}

pred apilado( $P_0, p : \text{Pila}\langle T \rangle, e : T$ ) {
  ( $\exists i : \mathbb{Z}$ ) ( $\text{esProximaPosicion}(P_0, k) \wedge p.\text{elemsEnPosicion} = \text{setKey}(P_0.\text{elemsEnPosicion}, e, i)$ )
}

pred esProximaPosicion( $p : \text{Pila}\langle T \rangle, i : \mathbb{Z}$ ) {
  ( $p.\text{elemsEnPosicion} = \langle \rangle \wedge i = 0$ )  $\vee_L$  ( $(\exists j : \mathbb{Z}) (\text{esPosicionDeTope}(p, j) \wedge i = j + 1)$ )
}

pred esPosicionDeTope( $p : \text{Pila}\langle T \rangle, i : \mathbb{Z}$ ) {
  ( $\forall t : T$ ) ( $t \in p.\text{elemsEnPosicion} \rightarrow_L p.\text{elemsInPosicion}[t] \leq i$ )
}

pred esTope( $p : \text{Pila}\langle T \rangle, res : T$ ) {
   $res \in \text{claves}(p.\text{elemsEnPosicion}) \wedge_L$ 
  ( $\exists i : \mathbb{Z}$ ) ( $\text{esPosicionDeTope}(p, i) \wedge p.\text{elemsEnPosicion}[res] = i$ )
}

pred desapilado( $P_0, p : \text{Pila}\langle T \rangle$ ) {
  ( $\exists t : T$ ) (
     $t \in \text{claves}(P_0.\text{elemsEnPosicion}) \wedge$ 
     $\text{esPosicionDeTope}(P_0, P_0.\text{elemsEnPosicion}[t]) \wedge$ 
     $p.\text{elemsEnPosicion} = \text{delKey}(P_0.\text{elemsEnPosicion}, t)$ 
  )
}

```

b) TAD Secuencia $\langle S \rangle$  {

```

obs elemsEnPosicion :  $\text{dicc}\langle \mathbb{Z}, S \rangle$ 

proc nuevaSecuencia() : Secuencia $\langle S \rangle$  {
  requiere { Verdadero }
  asegura {  $res.\text{elemsEnPosicion} = \langle \rangle$  }
}

proc vacia(in  $s : \text{Secuencia}\langle S \rangle$ ) : bool {
  requiere { Verdadero }
  asegura {  $res = \text{true} \leftrightarrow s.\text{elemsEnPosicion} = \langle \rangle$  }
}

proc longitud(in  $s : \text{Secuencia}\langle S \rangle$ ) :  $\mathbb{Z}$  {
  requiere { Verdadero }
  asegura {  $res = |s.\text{elemsEnPosicion}|$  }
}

proc obtener(in  $s : \text{Secuencia}\langle S \rangle$ , in  $i : \mathbb{Z}$ ) :  $S$  {
  requiere {  $\text{indiceEnRango}(s, i)$  }
  asegura {  $res = s.\text{elemsEnPosicion}[i]$  }
}

proc agregarAtras(inout  $s : \text{Secuencia}\langle S \rangle$ , in  $e : S$ ) {
  requiere {  $s = S_0$  }
  asegura {  $s.\text{elemsEnPosicion} = \text{setKey}(S_0.\text{elemsEnPosicion}, |S_0.\text{elemsEnPosicion}|, e)$  }
}

```

```

}

proc modificarPosicion(inout s : Secuencia⟨S⟩, in i : ℤ, in e : S) {
  requiere { s = S0 ∧ indiceEnRango(s, i) }
  asegura { s.elmsEnPosicion = setKey(S0.elmsEnPosicion, i, e) }
}

proc igualdad(in s, t : Secuencia⟨S⟩) : bool {
  requiere { Verdadero }
  asegura { res = true ↔ s.elmsEnPosicion = t.elmsEnPosicion }
}

pred indiceEnRango(s : Secuencia⟨S⟩, i : ℤ) { i ∈ s.elmsEnPosicion }
}

```

c) TAD Conjunto⟨C⟩ {

```

  obs elms : dicc⟨C, bool⟩
  proc nuevoConjunto() : Conjunto⟨C⟩ {
    requiere { Verdadero }
    asegura { res.elms = ⟨⟩ }
  }

  proc vacio(in c : Conjunto⟨C⟩) : bool {
    requiere { Verdadero }
    asegura { res = true ↔ c.elms = ⟨⟩ }
  }

  proc tamano(in c : Conjunto⟨C⟩) : ℤ {
    requiere { Verdadero }
    asegura { res = |c.elms| }
  }

  proc pertenece(in c : Conjunto⟨C⟩, in e : C) : bool {
    requiere { Verdadero }
    asegura { res = true ↔ e ∈ c.elms }
  }

  proc agregar(inout c : Conjunto⟨C⟩, in e : C) {
    requiere { c = C0 }
    asegura { c.elms = C0.elms ∪ {e} }
  }

  proc incluido(in c, d : Conjunto⟨C⟩) : bool {
    requiere { Verdadero }
    asegura { res = true ↔ claves(c.elms) ⊆ claves(d.elms) }
  }

  proc union(in c, d : Conjunto⟨C⟩) : Conjunto⟨C⟩ {
    requiere { Verdadero }
    asegura { res.elms = c.elms ∪ d.elms }
  }
}

```

Intersección y diferencia son iguales.

```

proc igualdad(in c, d : Conjunto⟨C⟩) : bool {
    requiere { Verdadero }
    asegura { res = true ↔ claves(c.elems) = claves(d.elems) }
}

```

- d) Igual que secuencia pero con agregarAdelante (como en el ejercicio 7) y sacando el elemento del medio de forma similar.

**Ejercicio 9.** Especificar TADs para las siguientes estructuras:

a) Multiconjunto⟨T⟩

También conocido como **multiset** o **bag**. Es igual a un conjunto pero con duplicados: cada elemento puede agregarse múltiples veces. Tiene las mismas operaciones que el TAD **Conjunto**, más una operación que indica la multiplicidad de un elemento (la cantidad de veces que ese elemento se encuentra en la estructura). Nótese que si un elemento es eliminado del multiconjunto, se reduce en 1 la multiplicidad.

Ejemplo:

```

c := new MultiConjunto<int>()

agregar(c, 1)
agregar(c, 1)
pertenece(c, 1) // devuelve true
multiplicidad(c, 1) // devuelve 2

sacar(c, 1)
pertenece(c, 1) // devuelve true
multiplicidad(c, 1) // devuelve 1

```

b) Multidict⟨K, V⟩

Misma idea pero para diccionarios: Cada clave puede estar asociada con múltiples valores. Los valores se definen de a uno (indicando una clave y un valor), pero la operación **obtener** debe devolver todos los valores asociados a una determinada clave.

Nota: En este ejercicio deberá tomar algunas decisiones. ¿Se pueden asociar múltiples veces un mismo valor con una clave? ¿Qué pasa en ese caso? ¿Qué parámetros tiene y cómo se comporta la operación **borrar**? Imagine un caso de uso para esta estructura y utilice su sentido común para tomar estas decisiones.

**Solución**

a) TAD MultiConjunto⟨T⟩ {

obs elemsConMult : dicc⟨T, ℤ⟩

```

proc conjuntoVacio() : MultiConjunto⟨T⟩ {
    requiere { Verdadero }
    asegura { |res.elemsConMult| = 0 }
}

```

```

proc igualdad(in c1, c2 : MultiConjunto⟨T⟩) : bool {
    requiere { Verdadero }
    asegura { res = true ↔ c1.elemsConMult = c2.elemsConMult }
}

```

```

}

proc agregar(inout c : MultiConjunto⟨T⟩, in e : T) {
  requiere { c = C0 }
  asegura {
    (e ∈ claves(C0.elemsConMult) ∧
      c.elemsConMult = setKey(C0.elemsConMult, e, C0.elemsConMult[e] + 1))
    ∨
    (e ∉ claves(C0.elemsConMult) ∧ c.elemsConMult = setKey(C0.elemsConMult, e, 1))
  }
}

proc pertenece(in c : MultiConjunto⟨T⟩, in e : T) : bool {
  requiere { Verdadero }
  asegura { res = true ↔ e ∈ c.elemsConMult }
}

proc eliminar(inout c : MultiConjunto⟨T⟩, in e : T) {
  requiere { c = C0 }
  asegura {
    e ∉ C0.elemsConMult ∧ c.elemsConMult = C0.elemsConMult
    ∨
    e ∈ C0.elemsConMult ∧L (
      C0.elemsConMult[e] > 1 ∧
      c.elemsConMult = setKey(C0.elemsConMult, e, C0.elemsConMult[e] - 1)
      ∨
      C0.elemsConMult[e] = 1 ∧
      c.elemsConMult = delKey(C0.elemsConMult, e)
    )
  }
}

proc multiplicidad(in c : MultiConjunto⟨T⟩, in e : T) : ℤ {
  requiere { Verdadero }
  asegura {
    e ∈ claves(c.elemsConMult) ∧ res = c.elemsConMult[e]
    ∨
    e ∉ claves(c.elemsConMult) ∧ res = 0
  }
}

```

```

b) TAD MultiDic⟨K, V⟩ {
  obs elemsConMult : dicc⟨K, conj⟨V⟩⟩
  proc nuevoDiccionario() : MultiDic⟨K, V⟩ {
    requiere { Verdadero }
    asegura { res.elemsConMult = ⟨⟩ }
  }

  proc igualdad(in d1, d2 : MultiDic⟨K, V⟩) : bool {
    asegura {
      res = true ↔ d1.elemsConMult = d2.elemsConMult
    }
  }
}

```

```

}

– Se acepta varias veces el mismo valor como parámetro, pero no se asocia repetido
proc definir(inout d : MultiDic⟨K, V⟩, in k : K, in v : V) {
  requiere { d = D0 }
  asegura {
    k ∈ D0.elemsConMult ∧
    d.elemsConMult = setKey(D0.elemsConMult, k, D0.elemsConMult[k] ∪ ⟨v⟩)
    ∨
    k ∉ claves(D0.elemsConMult) ∧
    d.elemsConMult = setKey(D0.elemsConMult, k, ⟨v⟩)
  }
}

proc obtener(in d : MultiDic⟨K, V⟩, ink : K) : conj⟨V⟩ {
  requiere { k ∈ d.elemsConMult }
  asegura { res = d.elemsConMult[k] }
}

proc está(in d : MultiDic⟨K, V⟩, in k : K) : bool {
  requiere { Verdadero }
  asegura { res ↔ k ∈ d.elemsConMult }
}

– Borra un valor de una clave, si es el único borra la clave
proc borrar(inout d : MultiDic⟨K, V⟩, in k : K, in v : V) {
  requiere { d = D0 }
  asegura {
    (k ∉ D0.elemsConMult ∨ v ∉ D0.elemsConMult[k]) ∧
    d.elemsConMult = D0.elemsConMult
    ∨
    (k ∈ D0.elemsConMult ∧ v ∈ D0.elemsConMult[k]) ∧L (
      |D0.elemsConMult[k]| > 1 ∧
      d.elemsConMult = setKey(D0.elemsConMult, k, D0.elemsConMult[k] – ⟨v⟩)
      ∨
      D0.elemsConMult[k] = 1 ∧
      d.elemsConMult = delKey(D0.elemsConMult, k)
    )
  }
}
}

```

### 3.4. TADs surtidos

**Ejercicio 10.** Especifique el TAD **Contadores** que, dada una lista de eventos, permite contar la cantidad de veces que se produjo cada uno de ellos. La lista de eventos es fija. El TAD debe tener una operación para incrementar el contador asociado a un evento y una operación para conocer el valor actual del contador para un evento.

- Modifique el TAD para que sea posible preguntar el valor del contador *en un determinado momento del pasado*. Si necesita conocer la fecha y hora actual, puede pasarla como parámetro a los procedimientos. Asuma que las fechas son números enteros (por ejemplo, la cantidad de segundos desde el 1 de enero de 1970).

**Solución**

Fecha ES  $\mathbb{Z}$

Evento ES string

- *Hago directamente la versión modificada. La no modificada sería casi igual pero el valor de las claves*
- *sería un entero. Incrementar le suma 1 a ese entero y obtener lo devuelve.*

TAD Contadores {

- *El valor del contador es la longitud - 1 (el último índice)*
- *Esta decisión implica que no se puede contar más de una vez en el mismo momento*

obs eventos : dicc(string, seq⟨Fecha⟩)

```
proc crearContadores(in eventos : seq⟨Evento⟩, in fechaActual : Fecha) : Contadores {  
  requiere { Verdadero }  
  asegura { (∀e : Evento) (e ∈ eventos ↔ (e ∈ res.eventos ∧L |res.eventos[e]| = 0)) }  
}
```

```
proc igualdad(in C1 : Contadores, in C2 : Contadores) : bool {  
  asegura { res = true ↔ C1.eventos = C2.eventos }  
}
```

```
proc incrementar(inout c : Contadores, in e : Evento, in fechaActual : Fecha) {  
  requiere {  
    c = C0 ∧  
    e ∈ c.eventos ∧L (|c.eventos[e]| = 0 ∨ fechaActual > ultimo(c.eventos[e]))  
  }  
  asegura { c.eventos = setKey(C0.eventos, e, C0.eventos[e] ++ ⟨fechaAct⟩) }  
}
```

```
proc obtenerEnFecha(in c : Contadores, in e : Evento, in fechaBuscada : Fecha, in fechaActual : Fecha) :  $\mathbb{Z}$  {  
  requiere { fechaBuscada < fechaActual ∧ e ∈ c.eventos ∧L |c.eventos[e]| > 0 }  
  asegura { esLaCuentaHastaLaFecha(c, e, res, fechaBuscada) }  
}
```

```
pred esLaCuentaHastaLaFecha(c : Contadores, e : Evento, res :  $\mathbb{Z}$ , fechaBuscada : Fecha) {  
  ((|c.eventos[e]| = 0 ∨L fechaBuscada < ultimo(c.eventos[e])) ∧ res = 0)  
  ∨L  
  (∃i :  $\mathbb{Z}$ ) (  
    1 ≤ i < |c.eventos[e]| ∧L  
    (c.eventos[e][i - 1] ≥ fechaBuscada < c.eventos[e][i] ∧ res = i)  
  )  
}
```

```
aux ultimo(s : seq⟨T⟩) : T = s[|s| - 1]
```

}

**Ejercicio 11.** Un caché es una capa de almacenamiento de datos de alta velocidad que almacena un subconjunto de datos, normalmente transitorios, de modo que las solicitudes futuras de dichos datos se atienden con mayor rapidez que si se debe acceder a los datos desde la ubicación de almacenamiento principal. El almacenamiento en caché permite reutilizar de forma eficaz los datos recuperados o procesados anteriormente.

Esta estructura tiene una interface de diccionario: guarda valores asociados a claves, con la diferencia de que los datos almacenados en un cache pueden *desaparecer* en cualquier momento, en función de diferentes criterios.

Especificar caches genéricos (con claves de tipo K y valores de tipo V) que respeten las operaciones indicadas y las siguientes políticas de borrado automático. Si necesita conocer la fecha y hora actual, puede pasarla como parámetro a los procedimientos o bien puede asumir que existe una función externa *horaActual()* :  $\mathbb{Z}$  que retorna la fecha y hora actual. Asuma que las fechas son números enteros (por ejemplo, la cantidad de segundos desde el 1 de enero de 1970).

TAD Cache⟨K, V⟩ {

```
proc esta(in c : Cache⟨K, V⟩, in k : K) : bool  
proc obtener(in c : Cache⟨K, V⟩, in k : K) : V  
proc definir(inout c : Cache⟨K, V⟩, in k : K, in v : V)  
}
```

a) FIFO o first-in-first-out:

El cache tiene una capacidad máxima (máximo número de claves). Si se alcanza esa capacidad máxima se borra automáticamente la clave que fue definida por primera vez hace más tiempo.

b) LRU o last-recently-used:

El cache tiene una capacidad máxima (máximo número de claves). Si se alcanza esa capacidad máxima se borra automáticamente la clave que fue accedida por última vez hace más tiempo. Si no fue accedida nunca, se considera el momento en que fue agregada.

c) TTL o time-to-live:

El cache tiene asociado un máximo tiempo de vida para sus elementos. Los elementos se borran automáticamente cuando se alcanza el tiempo de vida (contando desde que fueron agregados por última vez).

### Solución

```

a) TAD CacheFIFO( $K, V$ ) {
  obs capacidad :  $\mathbb{Z}$ 
  obs data :  $\text{dicc}(K, V)$ 
  – Claves en el orden en que los fueron agregadas
  obs claves :  $\text{seq}(K)$ 
  proc nuevoCache(in cap :  $\mathbb{Z}$ ) : CacheFIFO( $K, V$ ) {
    requiere { cap > 0 }
    asegura { res.capacidad = cap  $\wedge$  res.data =  $\langle \rangle$   $\wedge$  res.claves =  $\langle \rangle$  }
  }

  proc igualdad( $C_1, C_2$  : CacheFIFO( $K, V$ )) : bool {
    asegura {
       $C_1.\text{capacidad} = C_2.\text{capacidad}$ 
       $C_1.\text{data} = C_2.\text{data}$ 
       $C_1.\text{claves} = C_2.\text{claves}$ 
    }
  }

  proc esta(in c : CacheFIFO( $K, V$ ), in k :  $K$ ) : bool {
    asegura { res = true  $\leftrightarrow$  k  $\in$  c.data }
  }

  proc obtener(in c : CacheFIFO( $K, V$ ), in k :  $K$ ) :  $V$  {
    requiere { k  $\in$  c.data }
    asegura { res = c.data[k] }
  }

  proc definir(inout c : CacheFIFO( $K, V$ ), in k :  $K$ , in v :  $V$ ) {
    requiere { c =  $C_0$  }
    asegura {
       $c.\text{capacidad} = C_0.\text{capacidad} \wedge$ 
       $k \in C_0.\text{data} \wedge \text{redefinida}(C_0, c, k, v) \vee_L$ 
       $|C_0.\text{data}| < c.\text{capacidad} \wedge \text{claveAgregada}(C_0, c, k, v) \vee_L$ 
       $|C_0.\text{data}| = c.\text{capacidad} \wedge \text{claveAgregadaFIFO}(C_0, c, k, v)$ 
    }
  }

  pred redefinida( $C_0, c$  : CacheFIFO( $K, V$ ), in k :  $K$ , in v :  $V$ ) {
    c.data = setKey( $C_0.\text{data}, k, v$ )  $\wedge$  c.claves =  $C_0.\text{claves}$ 
  }

  pred claveAgregada( $C_0, c$  : CacheFIFO( $K, V$ ), in k :  $K$ , in v :  $V$ ) {
    c.data = setKey( $C_0.\text{data}, k, v$ )  $\wedge$  c.claves =  $C_0.\text{data.claves} ++ \langle k \rangle$ 
  }

```

```

}
pred claveAgregadaFIFO( $C_0, c : \text{CacheFIFO}\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {
   $c.data = \text{setKey}(\text{delKey}(C_0.data, C_0.data.claves[0]), k, v) \wedge$ 
   $c.claves = \text{tail}(C_0.data.claves) ++ \langle k \rangle$ 
}
}

```

b) TAD  $\text{CacheLRU}\langle K, V \rangle$  {

obs capacidad :  $\mathbb{Z}$

obs data :  $\text{dicc}\langle K, V \rangle$

– *Claves y su ultima hora de acceso*

obs ultimosAccesos :  $\text{dicc}\langle K, \mathbb{Z} \rangle$

proc nuevoCache(in  $cap : \mathbb{Z}$ ) :  $\text{CacheLRU}\langle K, V \rangle$  {

requiere {  $cap > 0$  }

asegura {  $res.capacidad = cap \wedge res.data = \langle \rangle \wedge res.ultimosAccesos = \langle \rangle$  }

}

proc igualdad( $C_1, C_2 : \text{CacheLRU}\langle K, V \rangle$ ) : bool {

asegura {

$C_1.capacidad = C_2.capacidad$

$C_1.data = C_2.data$

$C_1.ultimosAccesos = C_2.ultimosAccesos$

}

}

– *El enunciado te obliga a definir una interfaz fija para todos los cachés donde la operación obtener es un observador puro (in  $c : \text{Cache}$ ).*

– *Sin embargo, para que el algoritmo LRU (Last Recently Used) funcione, el simple acto de “obtener” un dato debe modificar la estructura interna del TAD para actualizar la “última hora de acceso” de esa clave. Si se mantiene la firma dictada por el enunciado (in ), no se puede modificar  $c.ultimosAccesos$ .*

proc esta(in  $c : \text{CacheLRU}\langle K, V \rangle$ , in  $k : K$ ) : bool {

asegura {  $res = \text{true} \leftrightarrow k \in c.data$  }

}

proc obtener(inout  $c : \text{CacheLRU}\langle K, V \rangle$ , in  $k : K$ ) :  $V$  {

requiere {  $k \in c.data \wedge c = C_0$  }

asegura {

$res = c.data[k] \wedge$

$c.capacidad = C_0.capacidad \wedge$

$c.data = C_0.data \wedge$

$c.ultimosAccesos = \text{setKey}(C_0.ultimosAccesos, k, \text{horaActual}())$

}

}

proc definir(inout  $c : \text{CacheLRU}\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {

requiere {  $c = C_0$  }

asegura {

$c.capacidad = C_0.capacidad \wedge$

$k \in C_0.data \wedge \text{redefinida}(C_0, c, k, v) \vee_L$

$|C_0.data| < c.capacidad \wedge \text{claveAgregada}(C_0, c, k, v) \vee_L$

$|C_0.data| = c.capacidad \wedge \text{claveAgregadaLRU}(C_0, c, k, v)$

)

}

```

}

pred redefinida( $C_0, c : \text{CacheLRU}\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {
   $c.data = \text{setKey}(C_0.data, k, v) \wedge c.ultimosAccesos = \text{setKey}(C_0.data.ultimosAccesos, k, \text{horaActual}())$ 
}

pred claveAgregada( $C_0, c : \text{CacheLRU}\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {
   $c.data = \text{setKey}(C_0.data, k, v) \wedge c.ultimosAccesos = \text{setKey}(C_0.data.ultimosAccesos, k, \text{horaActual}())$ 
}

pred claveAgregadaLRU( $C_0, c : \text{CacheFIFO}\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {
  ( $\exists kLRU : K$ ) (
     $\text{esClaveLRU}(C_0, kLRU) \wedge$ 
     $c.data = \text{setKey}(\text{delKey}(C_0.data, kLRU), k, v) \wedge$ 
     $c.ultimosAccesos = \text{setKey}(\text{delKey}(C_0.data.ultimosAccesos, kLRU), k, \text{horaActual}())$ 
  )
}

pred esClaveLRU( $C_0 : \text{CacheFIFO}\langle K, V \rangle$ , in  $kLRU : K$ ) {
   $kLRU \in C_0.ultimosAccesos \wedge_L (\forall k : K) (k \in C_0.ultimosAccesos \rightarrow_L C_0.ultimosAccesos[k] \leq C_0.ultimosAccesos[kLRU])$ 
}

```

c) TAD  $\text{CacheTTL}\langle K, V \rangle$  {

```

  obs TTL :  $\mathbb{Z}$ 
  obs data :  $\text{dicc}\langle K, V \rangle$ 
  – Claves y la hora que fueron agregadas
  obs horaAgregada :  $\text{dicc}\langle K, \mathbb{Z} \rangle$ 
  proc nuevoCache(in  $TTL : \mathbb{Z}$ ) :  $\text{CacheTTL}\langle K, V \rangle$  {
    requiere { TTL > 0 }
    asegura {  $res.TTL = TTL \wedge res.data = \langle \rangle \wedge res.horaAgregada = \langle \rangle$  }
  }

  proc igualdad( $C_1, C_2 : \text{CacheTTL}\langle K, V \rangle$ ) : bool {
    asegura {
       $C_1.TTL = C_2.TTL$ 
       $C_1.data = C_2.data$ 
       $C_1.horaAgregada = C_2.horaAgregada$ 
    }
  }

  proc esta(in  $c : \text{CacheTTL}\langle K, V \rangle$ , in  $k : K$ ) : bool {
    asegura {  $res = true \leftrightarrow k \in c.data \wedge_L \text{vigente}(c, k)$  }
  }

  proc obtener(in  $c : \text{CacheTTL}\langle K, V \rangle$ , in  $k : K$ ) :  $V$  {
    requiere {  $k \in c.data \wedge_L \text{vigente}(c, k)$  }
    asegura {  $res = c.data[k]$  }
  }

  proc definir(inout  $c : \text{CacheTTL}\langle K, V \rangle$ , in  $k : K$ , in  $v : V$ ) {
    requiere {  $c = C_0$  }
    asegura {

```

```

    c.TTL = C0.TTL ∧
    c.data = setKey(C0.data, k, v) ∧ c.horaAgregada = setKey(C0.data, k, horaActual())
  }
}

pred vigente(c : CacheTTL⟨K, V⟩, k : K) { horaActual() - c.horaAgregada[k] ≤ c.TTL }
}

```

**Ejercicio 12.** Especifique tipos para un robot que realiza un camino a través de un plano de coordenadas cartesianas (enteras), es decir, tiene operaciones para ubicarse en un coordenada, avanzar hacia arriba, hacia abajo, hacia la derecha y hacia la izquierda, preguntar por la posición actual, saber cuántas veces pasó por una coordenada dada y saber cuál es la coordenada más a la derecha por dónde pasó.

```

Coord ES struct ⟨x : ℤ, y : ℤ⟩
TAD Robot {
  proc ubicarse(inout r : Robot, in c : Coord)
  proc arriba(inout r : Robot)
  proc abajo(inout r : Robot)
  proc izquierda(inout r : Robot)
  proc derecha(inout r : Robot)
  proc coordActual(in r : Robot) : Coord
  proc másDerecha(in r : Robot) : ℤ
  proc cuantasVecesPaso(in r : Robot, in c : Coord) : ℤ
}

```

### Solución

```

Coord ES struct ⟨x : ℤ, y : ℤ⟩
TAD Robot {
  obs pos : Coord
  obs veces : dicc⟨Coord, ℤ⟩
  proc nuevoRobotEn(in p : Coord) : Robot {
    requiere { Verdadero }
    asegura { res.pos = p ∧ |res.vecas| = 1 ∧ res.vecas[p] = 1 }
  }

  proc arriba(inout r : Robot) {
    requiere { r = R0 }
    asegura { r.pos = posArriba(R0) ∧
      r.vecas = setKey(R0.vecas, posArriba(R0), vecesVisitado(R0, posArriba(R0)) + 1) }
  }

  proc abajo(inout r : Robot) {
    requiere { r = R0 }
    asegura { r.pos = posAbajo(R0) ∧
      r.vecas = setKey(R0.vecas, posAbajo(R0), vecesVisitado(R0, posAbajo(R0)) + 1) }
  }

  proc izquierda(inout r : Robot) {
    requiere { r = R0 }
    asegura { r.pos = posIzquierda(R0) ∧
      r.vecas = setKey(R0.vecas, posIzquierda(R0), vecesVisitado(R0, posIzquierda(R0)) + 1) }
  }

  proc derecha(inout r : Robot) {

```

```

    requiere {  $r = R_0$  }
    asegura {  $r.pos = posDerecha(R_0) \wedge$ 
 $r.vecas = setKey(R_0.vecas, posDerecha(R_0), vecesVisitado(R_0, posDerecha(R_0)) + 1)$  }
}

proc posicionActual(in  $r : Robot$ ) :  $Coord$  {
    requiere { Verdadero }
    asegura {  $res = r.pos$  }
}

proc cuantasVecesPaso(in  $r : Robot$ , in  $p : Coord$ ) :  $\mathbb{Z}$  {
    requiere { Verdadero }
    asegura {  $res = vecesVisitado(r, p)$  }
}

proc masDerecha(in  $r : Robot$ ) :  $Coord$  {
    requiere { Verdadero }
    asegura {
 $res \in r.vecas \wedge (\forall p : Coord) (p.inr.vecas \rightarrow_L p.x \leq res.x)$ 
    }
}

aux vecesVisitado( $r : Robot, p : Coord$ ) :  $\mathbb{Z} = \text{IfThenElse}(p \in r.vecas, r.vecas[p], 0)$ 
aux posArriba( $r : Robot$ ) :  $Coord = \langle x : r.pos.x, y : r.pos.y + 1 \rangle$ 
aux posAbajo( $r : Robot$ ) :  $Coord = \langle x : r.pos.x, y : r.pos.y - 1 \rangle$ 
aux posIzquierda( $r : Robot$ ) :  $Coord = \langle x : r.pos.x - 1, y : r.pos.y \rangle$ 
aux posDerecha( $r : Robot$ ) :  $Coord = \langle x : r.pos.x + 1, y : r.pos.y \rangle$ 
}

```

### 3.5. Ejercicios de parciales

**Ejercicio 13.** Queremos modelar el funcionamiento de un vivero. El vivero arranca su actividad sin ninguna planta y con un monto inicial de dinero.

Las plantas las compramos en un mayorista que nos vende la cantidad que deseemos pero solamente de a una especie por vez. Como vivimos en un país con inflación, cada vez que vamos a comprar tenemos un precio diferente para la misma planta. Para poder comprar plantas tenemos que tener suficiente dinero disponible, ya que el mayorista no acepta fiarnos.

A cada planta le ponemos un precio de venta por unidad. Ese precio tenemos que poder cambiarlo todas las veces que necesitemos. Para simplificar el problema, asumimos que las plantas las vendemos de a un ejemplar (cada venta involucra un solo ejemplar de una única especie). Por supuesto que para poder hacer una venta tenemos que tener *stock* de esa planta y tenemos que haberle fijado un precio previamente. Además, se quiere saber en todo momento cuál es el balance de caja, es decir, el dinero que tiene disponible el vivero.

- Indique las operaciones (procs) del TAD con todos sus parámetros.
- Describa el TAD en forma completa, indicando sus observadores y los requiere y asegura de las operaciones. Puede agregar los predicados y funciones auxiliares que necesite, con su correspondiente definición.
- ¿Qué cambiaría si supiéramos a priori que cada vez que compramos en el mayorista pagamos exactamente el 10% más que la vez anterior?

#### Solución

```

a) y b)
Planta ES  $\mathbb{Z}$ 
Dinero ES  $\mathbb{R}$ 
TAD Vivero {
    obs balance : Dinero
    – Este es el precio de venta al público, que es independiente del precio mayorista
    obs stock :  $\text{dicc}\langle \text{Planta}, \text{struct} \langle \text{cantidad} : \mathbb{Z}, \text{precio} : \text{Dinero} \rangle \rangle$ 
    proc abrirVivero(in  $montoInicial : \text{Dinero}$ ) : Vivero {

```

```

    requiere { montoInicial > 0 }
    asegura { res.balance = montoInicial ∧ res.stock = ⟨⟩ }
}

proc igualdad(in V1 : Vivero, in V2 : Vivero) : bool {
    asegura { res = true ↔ V1.balance = V2.balance ∧ V1.stock = V2.stock }
}

– Decidimos fijar el precio inicial con el precio mayorista, pero podríamos definirlo en 0 y sería válido
– Usar algún valor especial (negativo por ejemplo) para indicar “el precio no está definido” y luego
– usarlo en el requiere de vender() es una solución demasiado implementativa
proc compraMayorista(inout v : Vivero, in p : Planta, in cantidad : ℤ, in precio : Dinero) {
    requiere { v = V0 ∧ cantidad > 0 ∧ precio > 0 ∧ v.balance ≥ cantidad × precio }
    asegura {
        v.balance = V0.balance – cantidad × precio ∧ (
            seAgregoMercaderiaNueva(V0, v, p, cantidad, precio) ∨
            seAgregoStock(V0, v, p, cantidad, precio)
        )
    }
}

pred seAgregoMercaderiaNueva(V0, v : Vivero, p : Planta, cantidad : ℤ, precio : Dinero) {
    p ∉ V0.stock ∧ v.stock = setKey(V0.stock, p, ⟨cantidad, precio⟩)
}

pred seAgregoStock(V0, v : Vivero, p : Planta, cantidad : ℤ, precio : Dinero) {
    p ∈ V0.stock ∧
    v.stock = setKey(V0.stock, p, ⟨V0.stock[p].cantidad + cantidad, V0.stock[p].precio⟩)
}

proc ponerPrecio(inout v : Vivero, in p : Planta, in precio : Dinero) {
    requiere { v = V0 ∧ p ∈ v.stock ∧ precio > 0 }
    asegura {
        v.balance = V0.balance ∧
        v.stock = setKey(V0.stock, p, ⟨V0.stock[p].cantidad, precio⟩)
    }
}

proc vender(inout v : Vivero, in p : Planta) {
    requiere { v = V0 ∧ p ∈ v.stock ∧ v.stock[p].cantidad > 0 }
    asegura {
        v.balance = V0.balance + V0.stock[p].precio ∧
        v.stock = setKey(V0.stock, p, ⟨V0.stock[p].cantidad – 1, V0.stock[p].precio⟩)
    }
}

proc balance(in v : Vivero) : Dinero {
    requiere { Verdadero }
    asegura { res = v.balance }
}

```

- c) Si sabemos por cuánto cambia el precio mayorista en cada compra, sólo nos importa el precio inicial, y de ahí en más tenemos que calcular el precio en función del precio anterior.

Concretamente necesitamos:

- O bien agregar el precio mayorista al observador *stock* o bien agregar un nuevo observador con un diccionario que para cada Planta nos diga su precio mayorista
- Modificamos **encabezadoProcCompraMayorista**:
  - En el caso de una planta nueva
    - Definimos el precio mayorista inicial, además del precio de venta
    - El resto de la compra queda igual
  - En el caso de reponer stock
    - Actualizamos el precio de compra mayorista (multiplicando por 0.1)
    - Calculamos el balance en función de ese precio

**Ejercicio 14.** Se desea modelar mediante un TAD un videojuego de guerra desde el punto de vista de un único jugador. En el videojuego es posible ir a las tabernas y contratar mercenarios. Al contratarlo se nos informa el indicador de poder que tiene y el costo que tienen sus servicios. El poder de un mercenario siempre es positivo, sino nadie querría contratarlo. Los mercenarios no aceptan una promesa de pago, por lo que el jugador deberá tener el dinero suficiente para pagarlo. El jugador puede juntar la cantidad de mercenarios que desee para poder formar batallones. El poder de los batallones es igual a la suma del poder de cada uno de los mercenarios que lo componen. Cada mercenario puede pertenecer a un solo batallón

El jugador comienza con un monto de dinero inicial determinado por el juego. A su vez, comienza con un sólo territorio bajo su dominio. El objetivo del juego es conquistar la mayor cantidad de territorios posible para dominar el continente. Para ello, el jugador puede tomar uno de sus batallones y atacar un territorio enemigo. Al momento de atacar se conoce la fuerza del batallón enemigo. El jugador resulta vencedor si tiene más poder que el enemigo, en ese caso se anexa el territorio y se ganan 1000 monedas. Caso contrario, se debe pagar por la derrota una suma de 500 monedas. El jugador no puede ir a pelear si no tiene dinero para financiar su derrota.

Además, se desea saber en todo momento la cantidad de territorios anexados y el dinero disponible.

Se pide:

- a) Indique las operaciones (procs) del TAD con todos sus parámetros.
- b) Describa el TAD en forma completa, indicando sus observadores y los requiere y asegura de las operaciones. Puede agregar los predicados y funciones auxiliares que necesite, con su correspondiente definición.

### Solución

```

Mercenario, Batallón, Poder ES  $\mathbb{Z}$ 
TAD Juego {
  obs dinero :  $\mathbb{Z}$ 
  obs mercenarios :  $\text{dicc}(\text{Mercenario}, \text{Poder})$ 
  obs batallones :  $\text{dicc}(\text{Batallón}, \text{conj}(\text{Mercenario}))$ 
  obs territorios :  $\mathbb{Z}$ 
  proc iniciarJuego(in dineroInicial :  $\mathbb{Z}$ ) : Juego {
    requiere { dineroInicial > 0 }
    asegura {
      res.dinero = dineroInicial  $\wedge$ 
      res.mercenarios =  $\langle \rangle$   $\wedge$ 
      res.batallones =  $\langle \rangle$   $\wedge$ 
      res.territorios = 1
    }
  }
}

proc igualdad(in  $J_1$  : Juego, in  $J_2$  : Juego) : bool {
  asegura { res = true  $\leftrightarrow$ 
     $J_1$ .dinero =  $J_2$ .dinero  $\wedge$ 
     $J_1$ .mercenarios =  $J_2$ .mercenarios  $\wedge$ 
     $J_1$ .batallones =  $J_2$ .batallones  $\wedge$ 
     $J_1$ .territorios =  $J_2$ .territorios
  }
}

proc contratarGuerrero(inout  $j$  : Juego, in  $m$  : Mercenario, in  $p$  : Poder, in costo :  $\mathbb{Z}$ ) {

```

```

    requiere {
         $j = J_0 \wedge$ 
         $m \notin j.mercenarios \wedge$ 
         $costo \leq j.dinero \wedge$ 
         $poder > 0$ 
    }
    asegura {
         $j.dinero = J_0.dinero - costo \wedge$ 
         $j.mercenarios = setKey(J_0.mercenarios, m, p) \wedge$ 
         $j.batallones = J_0.batallones \wedge$ 
         $j.territorios = J_0.territorios$ 
    }
}

proc formarBatallon(inout  $j : Juego$ , in  $b : Batallón$ , in  $ms : conj\langle Mercenario \rangle$ ) {
    requiere {
         $j = J_0 \wedge$ 
         $b \notin j.batallones \wedge$ 
         $todosMercenariosContratados(j, ms)$ 
    }
    asegura {
         $j.dinero = J_0.dinero \wedge$ 
         $j.mercenarios = J_0.mercenarios \wedge$ 
         $j.batallones = setKey(J_0.batallones, b, ms) \wedge$ 
         $j.territorios = J_0.territorios$ 
    }
}

proc lucharPorTerritorio(inout  $j : Juego$ , in  $b : Batallón$ , in  $poderEnemigo : Poder$ ) {
    requiere {
         $j = J_0 \wedge$ 
         $b \in j.batallones \wedge$ 
         $j.dinero > 500 \wedge$ 
         $poderEnemigo > 0$ 
    }
    asegura {
         $j.mercenarios = J_0.mercenarios \wedge$ 
         $j.batallones = J_0.batallones \wedge$ 
        (
             $poderBatallon(j, b) \geq poderEnemigo \wedge$ 
             $j.dinero = J_0.dinero + 1000 \wedge j.territorios = J_0.territorios + 1$ 
        )  $\vee$  (
             $poderBatallon(j, b) < poderEnemigo \wedge j.dinero = J_0.dinero - 500$ 
        )
    }
}

proc dineroDisponible(in  $j : Juego$ ) :  $Dinero$  {
    requiere { Verdadero }
    asegura {  $res = j.dinero$  }
}

proc territoriosAnexados(in  $j : Juego$ ) :  $\mathbb{Z}$  {
    requiere { Verdadero }
    asegura {  $res = j.territorios$  }
}

pred todosMercenariosContratados( $j : Juego, ms : conj\langle Mercenario \rangle$ ) {  $ms \subseteq claves(j.mercenarios)$  }
aux poderBatallon( $j : Juego, b : Batallón$ ) :  $\mathbb{Z} =$ 
     $\sum_{m \in j.batallones[b]} j.mercenarios[m]$ 
}

```

**Ejercicio 15.** Se quiere organizar un viaje de estudios entre un grupo de estudiantes. Al definir el viaje se sabe qué personas viajan y los costos de los pasajes, así como el costo del combustible para un auto. Como el costo de pasajes es más alto que el del combustible, se quiere que todos viajen en auto, pero inicialmente no se sabe de cuántos se dispondrá.

A medida que se van confirmando autos disponibles se podrá asignar pasajeros a cada uno. Por razones de seguridad no se pueden asignar más de cuatro pasajeros a cada auto. Naturalmente el dueño de cada auto debe quedar asignado al suyo y nadie puede estar asignado a más de un auto. Una persona puede haber sido asignada a un auto pero luego confirmar el propio. Si pasa eso, el auto en el que iba a viajar esa persona antes de conseguir el suyo queda sin pasajeros para poder repartir todas esas personas.

Se quiere poder saber el costo total del viaje, es decir, el costo en pasajes sumado al costo en combustible para todos los autos.

- Indique las operaciones (procs) del TAD con todos sus parámetros.
- Describa el TAD en forma completa, indicando sus observadores y los requiere y asegura de las operaciones. Puede agregar los predicados y funciones auxiliares que necesite, con su correspondiente definición.

### Ejemplo

Condiciones iniciales:

- **Pasajeros:** Juan, Pedro, María, José, Laura, Carla, Manuel, Claudia
- **Costo de combustible:** \$40.000 por auto
- **Costo por pasaje:** \$30.000 por persona

Costo total actual:  $8 \times \$30.000 = \$240.000$

---

- Juan consigue su auto
- Se le asignan María, José, Laura y Manuel como pasajeros

Costo total actual:  $1 \times \$40.000 + 3 \times \$30.000 = \$130.000$

---

- José consigue su auto
- Se asignan a María, Manuel, Laura y Claudia al auto de Juan
- Se asignan a Pedro y Carla al auto de José

Costo total actual:  $2 \times \$40.000 = \$80.000$

### Solución

Persona, Auto ES  $\mathbb{Z}$

TAD ViajeDeEstudios {

```
obs viajeros : conj⟨Persona⟩
obs dueños : dicc⟨Auto, Persona⟩
obs asignaciones : dicc⟨Auto, conj⟨Persona⟩⟩
obs costoPasaje :  $\mathbb{Z}$ 
obs costoCombustible :  $\mathbb{Z}$ 
```

```
proc nuevoViaje(in viajeros : conj⟨Persona⟩, in costoPasaje, costoCombustible :  $\mathbb{Z}$ ) : ViajeDeEstudios {
  requiere { |viajeros| > 0 ∧ costoPasaje > 0 ∧ costoCombustible > 0 }
  asegura {
    res.viajeros = viajeros ∧
    res.costoPasaje = costoPasaje ∧
    res.costoCombustible = costoCombustible ∧
    res.dueños = ⟨⟩ ∧
    res.asignaciones = ⟨⟩
  }
}
```

```
proc igualdad(in  $V_1$  : ViajeDeEstudios, in  $V_2$  : ViajeDeEstudios) : bool {
```

```

    asegura { res = true  $\leftrightarrow$ 
       $V_1.viajeros = V_2.viajeros \wedge$ 
       $V_1.costoPasaje = V_2.costoPasaje \wedge$ 
       $V_1.costoCombustible = V_2.costoCombustible \wedge$ 
       $V_1.dueños = V_2.dueños \wedge$ 
       $V_1.asignaciones = V_2.asignaciones$ 
    }
  }
}

proc confirmarAuto(inout v : ViajeDeEstudios, in a : Auto, in dueño : Persona) {
  requiere {  $v = V_0 \wedge$  dueño  $\in v.viajeros \wedge a \notin v.dueños$  }
  asegura {
     $v.viajeros = V_0.viajeros \wedge$ 
     $v.costoPasaje = V_0.costoPasaje \wedge$ 
     $v.costoCombustible = V_0.costoCombustible \wedge$ 
     $v.dueños = setKey(V_0.dueños, a, dueño) \wedge$ 
    (
      estaAsignado( $V_0$ , dueño)  $\wedge$  seVacioElAutoEnQueViaja( $V_0, v, dueño$ )
    )  $\vee$  (
       $\neg$ estaAsignado( $V_0$ , dueño)  $\wedge v.asignaciones = V_0.asignaciones$ 
    )
  }
}

proc llenarAuto(inout v : ViajeDeEstudios, in a : Auto, in asignadosconj(Persona)) {
  requiere {
     $v = V_0 \wedge$ 
     $a \in v.dueños \wedge a \notin v.asignaciones$ 
     $|asignados| < 5 \wedge$ 
     $asignados \subseteq v.viajeros \wedge$ 
     $v.dueños[a] \in asignados$ 
    ningunoEstaAsignado( $v, asignados$ )
  }
  asegura {
     $v.viajeros = V_0.viajeros \wedge$ 
     $v.costoPasaje = V_0.costoPasaje \wedge$ 
     $v.costoCombustible = V_0.costoCombustible \wedge$ 
     $v.dueños = v.dueños \wedge$ 
     $v.asignaciones = setKey(V_0.asignaciones, a, asignados)$ 
  }
}

proc calcularCostoTotal(in v : ViajeDeEstudios) :  $\mathbb{Z}$  {
  requiere { Verdadero }
  asegura {  $res = costoTotalNafta(v) + costoTotalPasajes(v)$  }
}

pred estaAsignado( $v : ViajeDeEstudios, p : Persona$ ) {
  ( $\exists a : Auto$ ) ( $a \in v.asignaciones \wedge_L p \in v.asignaciones[a]$ )
}

pred seVacioElAutoEnQueViaja( $V_0, v : ViajeDeEstudios, p : Persona$ ) {
  ( $\exists a : Auto$ ) ( $a \in V_0.asignaciones \wedge_L p \in V_0.asignaciones[a] \wedge$ 
     $v.asignaciones = delKey(V_0.asignaciones, a)$ )
}

pred ningunoEstaAsignado( $v : ViajeDeEstudios, as : conj(Persona)$ ) {
  ( $\forall p : Persona$ ) ( $p \in as \rightarrow \neg estaAsignado(v, p)$ )
}

aux costoTotalNafta( $v : ViajeDeEstudios$ ) :  $\mathbb{Z} = |v.dueños| \times v.costoCombustible$ 
aux costoTotalPasajes( $v : ViajeDeEstudios$ ) :  $\mathbb{Z} =$ 
   $\sum_{p \in v.viajeros} \text{IfThenElse}(estaAsignado(v, p), 0, v.costoPasaje)$ 
}

```